

Rapport de corrections

1) Measurements — noms de principes passés au solveur Cypher

Contexte

Le module `Measurements` lance des vérifications en passant au solveur Cypher une liste de principes.

Ce qu'il y avait

```
private static final List<String> CYPHER_PRINCIPLE_NAMES_TO_TEST =  
List.of(  
    "LEGAL", "RIGHT_TO_ERASURE", "RIGHT_TO_ACCESS", "STORAGE_LIMITATION"  
);
```

Problème

Le mapping s'appuie sur `Issue.IssueType.fromString(...)` qui compare à `typeName` :

- `LEGAL` → `"legal"`
- `RIGHT_TO_ERASURE` → `"eraseCompliant"`
- `RIGHT_TO_ACCESS` → `"rightAccess"`
- `STORAGE_LIMITATION` → `"storageLimitation"`

Les constantes en MAJUSCULES ne correspondaient pas ⇒ `UNKNOWN` ⇒ requête Cypher `null` ⇒ **aucune vérif exécutée** (faux “tout conforme”) et **mesures faussées**.

Correction apportée

Utiliser les `typeName` de la source de vérité (l'enum) :

```
private static final List<String> CYPHER_PRINCIPLE_NAMES_TO_TEST =  
List.of(  
    Issue.IssueType.LEGAL.typeName,           // "legal"
```

```
Issue.IssueType.RIGHT_TO_ERASURE.typeName, // "eraseCompliant"
Issue.IssueType.RIGHT_TO_ACCESS.typeName,   // "rightAccess"
Issue.IssueType.STORAGE_LIMITATION.typeName // "storageLimitation"
);
```

Effet

- Les principes sont **correctement reconnus**.
 - Les requêtes Cypher **s'exécutent réellement**.
 - Les résultats/perfs dans `results*.csv` redeviennent **fiables**.
-

2) Issue — compteur d'ID non réinitialisé

Contexte

Chaque `Issue` affichée a un identifiant auto-incrémenté.

Ce qu'il y avait

```
private static final AtomicInteger counter = new AtomicInteger(0);

public Issue(...) {
    this.id = counter.incrementAndGet();
}
```

Jamais réinitialisé.

Problème

Plusieurs exécutions → IDs : `1..n` puis `n+1..` ⇒ confusion, logs peu lisibles, tests fragiles.

Correction apportée

Méthode de reset :

```
public static void resetCounter() { counter.set(0); }
```

-
- Appel en début d'exécution :

- `PrologSolver.solve(...) → Issue.resetCounter();`
- `SolverCypher.solve(...) → Issue.resetCounter();`

Effet

- Chaque run **repart à 1**.
- **Lisibilité** et **cohérence** accrues.
- Tests **déterministes**.

3) GraphDBToProlog — **purposes/3** mal généré (suffixe **_purposes**)

Problème

Export Neo4j → Prolog écrivait :

```
purposes('consent_X', 'phone_bob_purposes', [...]).
```

Le 2^e argument prenait le **nom de propriété** Neo4j au lieu du **nom de donnée**.

Correction

- Ignorer les clés méta : `name`, `type`, `category`, `consent_type`.
- Garder **uniquement** les propriétés finissant par `"_purposes"`.
- **Retirer** le suffixe pour reconstruire le nom de la donnée.
- Sérialiser proprement la liste Prolog via un helper `toPrologList(...)`.
- Gérer le cas spécial `_` (consentement joker) → `mandatory_consent`.

Extrait :

```
if (!key.endsWith("_purposes")) continue;
String data = key.substring(0, key.length() - "_purposes".length());
String dataArg = data.equals("_") ? "_" : "'" + data + "'";
```

```
String list = toPrologList(consentArtifact.get(key));
bw.write(String.format("purposes(%s,%s,%s).\n", consent, dataArg,
list));
```

Effet (Avant/Après)

- ❌ `purposes('consent_bob', 'phone_bob_purposes', ...)`
- ✅ `purposes('consent_bob', 'phone_bob', ...)`

Les règles Prolog retrouvent correctement les finalités.

4) PrologToGraphDB — parsing robuste des **purposes/3** (liste vide et commentaires en fin de ligne)

Problèmes traités

1. **Listes vides** : `purposes('c', 'd', [])`. était mal parsé en `[]` à cause d'un `split(",")` naïf.

Commentaires après un prédicat :

Exemple classique qui cassait le parseur :

```
purposes('consent_Alice_1', 'phoneNumber_Alice_1', ['sendAdSMS']). %
commentaire
```

- 2.

Corrections apportées

- **Listes vides** : si le contenu entre `[` et `]` est vide $\Rightarrow$ **tableau de longueur 0**.
On filtre tous les tokens vides **avant** d'enlever les quotes.
- **Commentaires inline** :
 - Nouveau stripping **sans toucher** au contenu entre crochets/quotes :
 - `stripInlineBlockComments(...)` remove `/* ... */` partout.

- `stripLineCommentOutsideQuotes(...)` supprime le % **seulement** s'il est **hors** quotes et **après** le point final `)`.
- Accumulation multi-lignes jusqu'à la fin du fait `" ) . "`.

Effet (Avant/Après)

- Avant : `[]` devenait `[]` ❌
Après : `[]` reste `[]` ✅
- Les lignes du style `fact(...). % note` sont **acceptées** sans ambiguïté.

5) PrologToGraphDB — parser de tête robuste, normalisation & logs propres

Contexte

Le fichier de graphes contient des titres/bannières, puces, etc. (ex. `A-1) ACCESS – Alice ...`).

Problème

Ces lignes “ornementales” pouvaient être vues comme des prédicats “bizarres” et polluer les logs.

Corrections

- **Normalisation :**
 - Suppression des caractères non alphabétiques en tête de ligne (puces, BOM/NBSP...).
 - Collapse des espaces successifs.

Extraction de la tête avec une **regex** robuste :

```
^\s*([A-Za-z_][A-Za-z0-9_]*)\s*\((
```

-
- **Ignorer** silencieusement ce qui n'est pas un fait Prolog.
Avertir uniquement si ça **ressemble** à un prédicat inconnu.

Effet

- Plus de *faux positifs* dans les logs.
 - Lisibilité des avertissements.
-

6) PrologToGraphDB — marquage des données personnelles & deletes implicites

Contexte

Pour calquer la sémantique Prolog en Cypher, on a besoin :

- d'identifier clairement les **données personnelles**,
- de **synthétiser** certains deletes à partir de `notAvailable/2`.

Corrections

- Quand `wasGeneratedBy(..., 'personal data', TG)` :
 - on tague l'artefact `type='personal_data'`,
 - on lui assigne un **ordre stable** `personal_seq` (séquence interne),
 - on calcule une clé `dp_key = sanitize(artifact) + "_purposes"`.
- `notAvailable(D, TU)` :
 - si **aucun** delete réel à ce TU, on **crée** un process synthétique `__implicit_delete__...` (action `delete`) et une relation `(:Process)-[:used {TU}]->(D)` marquée `synthetic=true`.

Effet

- Les **ancêtres personnels** sont stables (utile pour émuler `once/1`).
 - Les cas “donnée plus dispo” sont **compatibles** avec la logique temps-limite.
-

7) PrologToGraphDB — matérialisation des liens dérivés & racines personnelles

Contexte

Certaines règles Prolog infèrent la provenance.

Corrections

- Matérialisation : si `dOut wasGeneratedBy p et p used dIn` $\Rightarrow$ `dOut wasDerivedFrom dIn`, avec `T = coalesce(TG, TU)`.
- Fermeture 2 sauts : `out -> mid -> in2` $\Rightarrow$ `out -> in2 (ctx='derived2')`.
- Lien `hasPersonalRoot` auto-bouclé : `(dp:Artifact {type:'personal_data'})-[ :hasPersonalRoot ]->(dp)`.

Effet

- Parcours d'ascendance **efficace** côté Cypher.
- Cohérence avec la sémantique Prolog.

8) SolverCypher — réécriture complète de la requête RIGHT_TO_ACCESS

Contexte

Deux scénarios doivent être repérés :

1. **Jamais répondu** : une demande d'accès sans envoi ultérieur.
2. **Trop tard** : réponse envoyée **après** le délai (ex. `> 1000ms`).

Problèmes des versions antérieures

- Accrochage trop strict/fragile entre la demande (`askDataAccess`) et la réponse (`sendData`) : certains graphes valides n'étaient pas couverts.
- Conditions temporelles incomplètes ou évaluées au mauvais endroit.

Correction apportée (logique de la requête)

- On part d'un **événement de demande** : artefact `data_request_*` généré par `askDataAccess(...)` avec `T_req`.
- **Branche 1 — Jamais répondu** : il n'existe pas de `sendData` :
 - `ni (:Process {action:'sendData'})-[ :used {TU} ]->(data_request_*)` (cas "on renvoie la demande"),
 - `ni (:Process {action:'sendData'})-[ :wasGeneratedBy {TG} ]->(data_report_*)` **associé** à cette même demande,
 - et la fenêtre `now - T_req > limit` est dépassée.
- **Branche 2 — Trop tard** : il **existe** un `sendData`, mais `T_resp - T_req > limit`.
On couvre **les deux formes** usuelles :
 - `sendData used data_request_*` (horodatage TU),
 - `data_report_* wasGeneratedBy sendData` (horodatage TG).

Concrètement, la requête :

- isole proprement la **demande**,
- trouve (ou non) la **réponse** quel que soit le pattern utilisé dans le graphe,
- compare les temps **au bon endroit**,
- remonte des issues explicites (qui, quoi, quand).

Effet

- Les cas "**jamais répondu**" et "**trop tard**" sont correctement détectés.
- Les **logs** correspondent aux scénarios du graphe (Alice jamais répondu, Bob trop tard).
- Parité avec la sémantique Prolog sur l'accès.

9) Measurements — bench *par principe* + CSV dédiés

Contexte

Le bench global mélangeait tout dans un seul `results.csv`.

Problème

Difficile d'analyser par principe (legal vs erase vs access vs storage).

Correction apportée

- Liste **PRINCIPLES** et map des **requêtes Prolog** par principe.
- **CSV par principe** :
 - `results_legal.csv`
 - `results_eraseCompliant.csv`
 - `results_rightAccess.csv`
 - `results_storageLimitation.csv`
- On conserve aussi le CSV global.
- Warm-up, logs, *skip files* : **inchangés**.

Effet

- Comparaison **fine** par principe **sans perdre** la vue globale.
- Fichiers prêts à être graphed/analysés individuellement.

10) PrologSolver — détection fiable du principe & logs propres

Problème

`fromString` recevait parfois une **requête complète** (`legal(P,D,...)`) au lieu du simple nom (`legal`) ⇒ `UNKNOWN` dans les logs, et chemins de règles inconsistants.

Correction

- Extraction du **prédicat** via regex (`extractPredicate(...)`) :

- `legal(P,D,...) → legal`
 - `legal. → legal`
 - `legal → legal`
- Si l'entrée contient des parenthèses ⇒ on **garde** la requête telle quelle. Sinon ⇒ on **fabrique** la requête via un template par principe.
- **Silence** des warnings Prolog non utiles :
 - `set_prolog_flag(single_var_warnings, false).`
 - Hook `message_hook` pour ignorer le niveau "informational".

Effet

- Plus de `UNKNOWN` dans les logs.
- Chargement des **bons fichiers de règles** et **bons templates d'issues**.
- Logs **propres** pendant les benches.

11) Storage Limitation — alignement exact Prolog ⇔ Cypher (rappel synthétique)

(Cette partie a été faite plus tôt ; on la rappelle ici car elle est structurante.)

Points clés

- **Choix du DP** comme Prolog `once/1` : tri par `personal_seq` croissant (**ordonnancement stable**) pour émuler le "premier ancêtre personnel" trouvé.
- **Fenêtre temporelle** : filtrage **en amont** sur `u.TU <= cutoff` (équivalent à "hors délai").
- **Delete acceptable** : test strict `(ud.TU - u.TU) < limit` **sans** forcer `ud.TU >= u.TU` (comme Prolog).
- **Refactor CALL / WITH** : WITH d'import **simple** sans WHERE dans les sous-requêtes (sinon `Importing WITH ...`).
- **Index** recommandés/créés :

- `(a:Artifact) ON (a.name), (a.type), (a.personal_seq)`
- `(p:Process) ON (p.action)`
- `()-[u:used]-() ON (u.TU)`

Effet

- **Parité** Prolog/Cypher sur tous les graphes de test.
- **Perfs** stables, plans déterministes.
- Fin des erreurs `Importing WITH ....`

12) Default purposes — alignement & logs *une seule fois*

Contexte

Comparer ce que voient Cypher vs Prolog sur les finalités par défaut.

Correction

- Log **unique par graphe** (flags internes pour ne pas spammer).
- Côté Prolog : récupération **de la liste** via `setof/3` et conversion en `List<String>` (pas de `writeln` absorbé).

Effet

- Deux lignes claires par graphe :
 - `Default purposes from graph: [...]`
 - `Default purposes from Prolog: [...]`
- Divergences immédiatement visibles.

13) Détails “Qualité de données” côté Neo4j

Ce qui a été renforcé

- `ensureArtifact/Process/Agent` en **MERGE** systématique.
- `sanitizePropKey(...)` :
 - on garde lettres/chiffres/underscore,
 - on remplace le reste par `_`,
 - on évite un début par chiffre.
- Pour `purposes(...)` :
 - écriture **exacte** de `[]` quand la liste est vide,
 - fusion **sans doublon nul** quand on cumule des lignes.

Effet

- **Homogénéité** des propriétés.
- Données prêtes pour des requêtes **fiables** (sans surprises syntaxiques).

14) LEGAL (Prolog) — optimisation complète, pourquoi c'était lent, ce qui a changé, et l'impact

La règle `legal/6` doit décider si un **usage de donnée personnelle** à l'instant `T` est **couvert par un consentement valide** (ou par une finalité par défaut). Dans les anciens fichiers, les mêmes sous-preuves étaient ré-évaluées des millions de fois, ce qui faisait exploser le coût CPU.

```
Total time: 758.766 seconds
=====
Predicate                                Box Entries =    Calls+Redos    Time
=====
used/4                                   8,384,918,646 =8,384,917,000+1,646    52.1%
wasControlledBy/5                        8,360,270,510 =31,819,602+8,328,450,908    34.0%
nextConsent/3                           8,370,701,198 =31,813,553+8,338,887,645    11.4%
lists:nth_gen/5                          34,139,262 =19,397,990+14,741,272    0.6%
consentFoundOk/4                        8,344,261,938 =    780+8,344,261,158    0.5%
wasGeneratedBy/4                       36,364,890 =21,320,221+15,044,669    0.4%
lastConsent/1                          8,353,061,159 =    5,327+8,353,055,832    0.3%
succ/2                                  28,183,478 =28,183,478+0    0.2%
nth0/3                                  34,139,262 =19,397,990+14,741,272    0.2%
```

Dans ce profil, on voit :

- `used/4` et `wasControlledBy/5` dominant (52% et 34% du temps) car on **énumérait massivement** tous les couples possibles avant de filtrer.
- `nextConsent/3` (11.4%) et `lastConsent/1` (0.3%) sont recalculés encore et encore.
- `nth0/3` et `lists:nth_gen/5` apparaissent (0.8% cumulé) **juste** pour tester l'appartenance à une liste (!).

Le problème racine

1. **Ordre de liaison défavorable** dans `consent/4` :

```
consent(C,D,PU,T) :-
```

```
    wasControlledBy(P1,_, 'owner',_,_),
```

```
    wasGeneratedBy(C,P1, 'consent',T),
```

```
    isPurpose(PU,D,C).
```

→ On parcourt d'abord **tous** les processus "owner" et **tous** les consentements, **avant** de restreindre par `(D,PU)` via `isPurpose/3`. C'est un **cartésien** coûteux sur de gros graphes.

2. **Aucune mémorisation** : `nextConsent/3`, `lastConsent/1`, `used/4` $\bowtie$ `action/2`, etc. sont re-prouvés à l'identique des milliers de fois.
3. **Test d'appartenance inefficace** : `isPurpose/3` utilisait `nth0/3`, qui génère des index via `succ/2` et `lists:nth_gen/5` → inutilement verbeux pour... vérifier qu'un élément est dans une liste.
4. **Chemin "par défaut" déclenché tard** : on appelait quand même la grosse machine `consentFoundOk/4` même si la finalité était dans les **default purposes** (le court-circuit ne se faisait qu'à l'intérieur du prédicat).

Les changements apportés (version "patched", sans couper la complétude)

1) Réordonner les liaisons pour filtrer tôt

```

consent(C, D, PU, T) :-
    isPurpose(PU, D, C),                % (D,PU) -> filtre d'abord sur
C !                                     % C !

    wasGeneratedBy(C, P1, 'consent', T),

    owner_proc(P1).

```

En liant **d'abord C** via `(D,PU)`, on ne passe à `wasGeneratedBy/4` et `wasControlledBy/5` que pour **les consentements pertinents**. Ça coupe l'explosion combinatoire.

2) Tabling ciblé (“mini-index” en mémoire)

```

:- table owner_proc/1.

owner_proc(P) :- wasControlledBy(P, _, 'owner', _, _).

:- table used_consent/2, revoke_event/2.

used_consent(P, C) :- used(P, C, 'consent', _).

revoke_event(C, T) :- used(_P, C, 'revokeConsent', T).

:- table used_action/4.

used_action(P, D, PU, T) :- used(P, D, _, T), action(P, PU).

:- table nextConsent/3, lastConsent/1.

```

- `owner_proc/1` évite de balayer `wasControlledBy/5` à répétition.
- `used_action/4` matérialise la jointure `used` $\bowtie$ `action` **une seule fois**, puis la réutilise partout.
- `nextConsent/3` et `lastConsent/1` sont **mémorisés** : d'énormes gains quand on vérifie une même fenêtre `(C,T)` pour des dizaines d'usages.

3) Appartenance de liste simplifiée

```

isPurpose(PU, D, C) :-
    purposes(C, D, Q),

```

```
member(PU, Q).                % remplace nth0/3
```

`member/2` suffit. On supprime `nth0/3`, `lists:nth_gen/5`, et toute la micro-machinerie d'index inutiles. Résultat : moins d'appels, moins de backtracking.

4) Court-circuit des finalités par défaut au plus haut niveau

```
legal(P, D, PU, T, DP, C) :-  
    used_action(P, D, PU, T),  
    once(isPersonalP(D, DP)),  
    \+ default_purpose(PU),          % si défaut → on évite tout le  
    chemin consent  
    \+ consentFoundOk(C, DP, PU, T).
```

Si `PU` est par défaut, on **n'évalue pas** la machine de consentement (qui était l'essentiel du coût). Avant, on entrait quand même dans `consentFoundOk/4` pour ensuite sortir très vite — mais **trop tard**.

5) Fenêtre de validité restructurée

```
consentFoundOk(C, D, PU, T) :-  
    consent(C, D, PU, TG), TG < T,  
    (    nextConsent(C, _C1, TG1), TG1 > T, !  
    ;    lastConsent(C),  
        (    \+ revoke(C, _), !  
        ;    revoke(C, TU), TU > T, !  
        )  
    ).
```

- On valide **une seule** paire (**C**, **TG**) puis on teste la **fenêtre** soit via **nextConsent**, soit via **lastConsent** + **revoke**.
- Les **coupes (!) locales** sont là **uniquement** pour arrêter la recherche dès qu'on a trouvé une preuve valable pour cette fenêtre — sans compromettre d'autres solutions éventuelles plus haut.

Pourquoi ça accélère concrètement

- **Moins de combinaisons** : (**D**, **PU**) → **C** avant les parcours de graphes (au lieu d'après).
- **Sous-preuves mises en cache** (tabling) : plus de recalcul de **nextConsent/3**, **lastConsent/1**, **owner_proc/1**, **used_action/4** identiques.
- **Zéro overhead inutile** : on remplace **nth0/3** par **member/2**.
- **Court-circuit réel des defaults** : aucune descente dans la logique consent quand **PU** est par défaut.

Résultat : les hotspots **disparaissent** ou sont **divisés** par un ordre de grandeur. Les tests montrent que, sur les graphes réalistes, **Prolog devient plus rapide que Cypher** pour **legal**.

Mesures (extrait CSV) — Prolog plus rapide sur **legal**

```
FILE_NAME,TYPE,NB_USERS,SIZE_KO,NODE_COUNT,USER_COUNT,SOLVER,ISSUE_COUNT,SOLVING_TIME_MS
20_45_W.pl,W,21,156,353,21,Prolog,376,446
20_45_W.pl,W,21,156,353,21,Cypher,376,2156
20_46_SN.pl,SN,21,169,354,21,Prolog,385,420
20_46_SN.pl,SN,21,169,354,21,Cypher,385,1632
20_50_PS.pl,PS,21,165,327,21,Prolog,341,366
20_50_PS.pl,PS,21,165,327,21,Cypher,341,1584
2_3_PS.pl,PS,3,12,55,3,Prolog,24,17
2_3_PS.pl,PS,3,12,55,3,Cypher,24,10
2_3_SN.pl,SN,3,10,52,3,Prolog,35,14
2_3_SN.pl,SN,3,10,52,3,Cypher,35,8
2_3_W.pl,W,3,17,68,3,Prolog,38,19
2_3_W.pl,W,3,17,68,3,Cypher,38,17
```

Lecture rapide :

- Sur les **gros graphes** (20_*), Prolog tombe à **~400–450 ms** quand Cypher reste à **1.5–2.1 s**.
- Sur les **micro-graphes** (2_*), la latence fixe de Prolog et Cypher est comparable ; Cypher peut rester légèrement devant (10–17 ms vs 14–19 ms) — sans enjeu pratique.

15) crash “controller null” dans **CONSENT_QUERY** et corrections

1) Version initiale (avant les changements)

Où ? Dans la requête LEGAL (**CONSENT_QUERY**), on calculait la racine personnelle “à la volée” avec un sous-bloc :

```
CALL {  
  WITH d  
  MATCH pth = allShortestPaths((d)-[:wasDerivedFrom*0..]->(dp:Artifact  
{type:'personal_data'}))  
  WITH dp, length(pth) AS L, coalesce(dp.personal_seq,  
9223372036854775807) AS seq  
  ORDER BY L ASC, seq ASC, dp.name ASC  
  LIMIT 1  
  RETURN dp  
}
```

...puis on enchaînait sur la logique de consentement (incluant le **MATCH** (pGen)-[:wasControlledBy {ctx:'owner'}]->(subj)).

Le plantage exact observé (sur graphes SN) :

```
Cannot invoke "org.neo4j.cypher.internal.runtime.pipelined.state  
.AbstractArgumentStateMap$StateController.increment(int)"  
because "controller" is null
```

Pourquoi ?

- Le planneur **pipelined** de Neo4j a un bug/instabilité avec ce pattern : var-length + **allShortestPaths** à l'intérieur d'un **CALL{ }** dans une requête plus large qui continue avec des **MATCH** contraignants (ici le “owner”).
 - Dans certains cas (consentements sans **wasControlledBy/owner** ou topologies particulières), l'état interne *controller* du pipeline n'est pas instancié → **NPE côté moteur** (= le message ci-dessus).
 - Ce n'est **pas** un “NULL Cypher normal” (qu'on pourrait gérer), c'est un **crash du runtime pipelined**.
-

2) Premier contournement : sortir ce calcul du solver → matérialiser dans le loader

Ce qu'on a fait :

- On a **déplacé** le calcul de la racine personnelle dans `PrologToGraphDB` :
 - Matérialisation des liens `wasDerivedFrom` (direct & “derived2”).
 - **Self-link** sur chaque DP : `(dp)-[:hasPersonalRoot]->(dp)`.
 - Choix d'**une** racine par artefact via shortest path.
- Dans `CONSENT_QUERY`, on n'a plus besoin de shortest paths : on fait simplement `MATCH (d)-[:hasPersonalRoot]->(dp)`.

Effet :

- On **retire** la partie fragile du solver → plus de crash “**controller null**” à cet endroit.
- LEGAL (Cypher) devient **plus rapide** (moins de travail pendant le solve).

3) Deuxième tentative (dans le loader) : remplacer `CALL {}` par `OPTIONAL MATCH + head(collect(...))`

But. Éviter `CALL {}` (puisqu'on l'avait vu heurter le runtime pipelined).

Requête utilisée (schéma) :

```
MATCH (d:Artifact)
OPTIONAL MATCH pth =
allShortestPaths((d)-[:wasDerivedFrom*0..]->(dp:Artifact
{type:'personal_data'}))
WITH d, dp, length(pth) AS L, coalesce(dp.personal_seq,
9223372036854775807) AS seq
ORDER BY d, L ASC, seq ASC, dp.name ASC
WITH d, head(collect(dp)) AS dp
MERGE (d)-[:hasPersonalRoot]->(dp)
```

Résultat :

- Le plantage **runtime pipelined** n'apparaît plus.

- **Mais** lorsque **aucun chemin** personnel n'existe, `head(collect(dp))` donne **NULL**, et le `MERGE (d)-[:hasPersonalRoot]->(dp)` **échoue** (Neo4j refuse de créer une relation vers un nœud NULL).
- Côté perfs, cette approche s'est révélée **moins rapide** : l'agrégation + tri global par `OPTIONAL + collect` coûte plus cher qu'un petit `CALL{... LIMIT 1}`.

4) Solution finale adoptée : revenir à `CALL{}` dans le loader et forcer

CYPHER runtime=interpreted

Requête (schéma) :

```
CYPHER runtime=interpreted
MATCH (d:Artifact)
CALL {
  WITH d
  MATCH pth = allShortestPaths((d)-[:wasDerivedFrom*0..]->(dp:Artifact
{type:'personal_data'}))
  WITH dp, length(pth) AS L, coalesce(dp.personal_seq,
9223372036854775807) AS seq
  ORDER BY L ASC, seq ASC, dp.name ASC
  LIMIT 1
  RETURN dp
}
MERGE (d)-[:hasPersonalRoot]->(dp)
```

Pourquoi ça règle tout :

- `CALL{... LIMIT 1}` n'émet aucune ligne si aucun `dp` → **pas de MERGE vers NULL** → plus de crash de type "node missing / UNNAMED / dp@...".
- Le hint `runtime=interpreted` désactive le moteur **pipelined** sur cette portion (seule), évitant l'exception "controller is null".
- Le coût du mode interprété est **négligeable** ici (exécuté une fois au chargement), et **plus rapide** que la variante `OPTIONAL + collect`.

Logique conservée à l'identique :

- Même tri de tie-break (`distance` → `personal_seq` → `name`) → mêmes racines élues qu'avant.

- On n'exclut **pas** `derived/derived2` (c'est bien ce que vous vouliez : ces arêtes proviennent du loader et font partie de la provenance attendue).
-

5) Effets mesurés & stabilité

- **Plus de crash** "Cannot invoke ... `StateController.increment` ... because `controller` is null".
 - **LEGAL (Cypher) plus rapide** qu'avant : le solver consomme `hasPersonalRoot` déjà prêt.
 - Vos benches montrent le gap (ex. `200_3_SN.pl` : Cypher ~**28 315 ms** vs Prolog ~**68 495 ms** pour LEGAL).
 - La requête LEGAL est **plus simple et robuste** (plus de var-length à l'intérieur).
-

6) Récap chronologique ultra-court

1. **Initial** : `CALL{ allShortestPaths }` dans `CONSENT_QUERY` → bug du runtime `pipelined Neo4j` → **crash** "controller null".
2. **Déplacement** : calcul de racine **dans le loader** + `hasPersonalRoot` → le solver n'a plus le pattern fragile.
3. **Essai sans CALL** : `OPTIONAL MATCH + head(collect)` → `dp NULL` possible → `MERGE` plante et perfs ↓.
4. **Final** : revenir à `CALL{ }` dans le loader + `CYPHER runtime=interpreted` → **zéro crash, perfs OK, logique inchangée**.